Bonus: POCing Alpine Vulnerabilities

Session Outline

- Brief introduction on how to reproduce a vulnerability within Alpine
- When to PoC, What information is needed
- How to build the package within Alpine to make PoCing easier

Follow Along! There will be homework after this session

Current Triage Process

Things to consider before adding an upstream vulnerability to the VMS

- Upstream Maintainer Disputed
- Disputed by the Debian Security team
- Not enough information provided by the Source

If Debian Security team considers a vuln to an issue and are working on a fix, we should add it too

No need for a Proof of Concept for Linux Vulns in most cases

Scenarios when trying to PoC

- Scenario 1: Published with Proof of Concept/Exploit
- Scenario 2: Published with crash Information and information regarding vulnerable code
- Scenario 3: Published with only vulnerable code

It might not be possible to provide full assurance that the vulnerability might exist within Alpine, However this is the case for many vulnerabilities within the Linux ecosystem.

Vulnerability	Verifiable	Reproducible	Reasoning
https://snyk.io/vuln/SNYK-UPSTREAM-SAMBA-536842	Yes	No	Scenario 3
https://snyk.io/vuln/SNYK-ALPINE310-SQLITE-454803	Yes	Yes	Scenario 1
https://snyk.io/vuln/SNYK-ALPINE37-IMAGEMAGICK6-495333	Yes	Yes	Scenario 2
https://snyk.io/vuln/SNYK-UBUNTU1604-SOX-345474	Yes	Yes	Scenario 2

Scenario 2 - A Working Example

Vulnerability: https://snyk.io/vuln/SNYK-UBUNTU1604-SOX-345474

Package: **SoX 14.4.2**

Vulnerability: Integer Overflow

Reference: https://sourceforge.net/p/sox/bugs/320, https://security-tracker.debian.org/tracker/CVE-2019-8355

size != length in C

Data type	Size(bytes)	Range	Format string
Char	1	128 to 127	%c
Unsigned char	1	0 to 255	%c
Short or int	2	-32,768 to 32,767	%i or %d
Unsigned int	2	0 to 65535	%u
Long	4	-2147483648 to 2147483647	%ld
Unsigned long	4	0 to 4294967295	%lu
Float	4	3.4 e-38 to 3.4 e+38	%f or %g
Double	8	1.7 e-308 to 1.7 e+308	%lf
Long Double	10	3.4 e-4932 to 1.1 e+4932	%lf

Integer Overflow

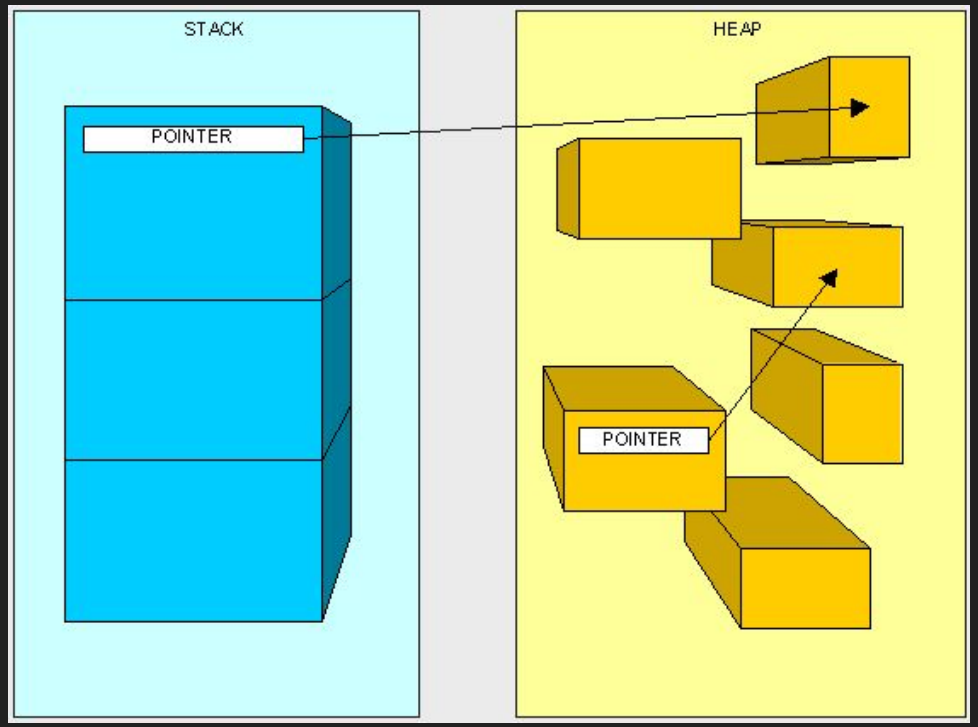
- 8-bit signed integer in C has a maximum value of 127 and a minimum value of -128.
- If a programmer stores the value 127 in such a variable and adds 1 to it, the result should be 128.
- However, this value exceeds the maximum for this integer type, so the interpreted value will “wrap around” and become -128.

Can lead to many issues, here it is a heap overflow.

https://github.com/chirlu/sox/blob/sox-14.4.0/src/xmalloc.h#L31

P R N D L2 L1
P R N D L2 L1
P R N D L2 L1

Heap



Heap Overflow

- Heap is a memory region allotted to every program.
- Unlike stack, heap memory can be dynamically allocated.
- This means that the program can 'request' and 'release' memory from the heap segment whenever it requires.
- Heap memory blocks are called chunks
- A chunk can be busy or free
- Very surgical memory corruption bugs relate to the heap.
- Exploitation involves overwriting fixed function pointers to point to attacker code

https://github.com/chirlu/sox/blob/sox-14.4.0/src/remix.c#L234

Things we are trying to Answer

- Exact input that triggered the crash: https://sourceforge.net/p/sox/bugs/320
- What vulnerability was triggered: **Integer Overflow**
- Type/Category of the crash: **Heap Overflow**

Where is the vulnerable code inside the package: *xmalloc.h*, ***lsx_valloc*** - Integer Overflow, heap-buffer-overflow is triggered in ***remix.c line 237***

- Exploitable: Not possible to answer in many cases

Next Steps

- Build the correct Alpine Environment
- Compile/Install the vulnerable version of the package
- Use a debugger and payload to verify the vulnerability

Building a Vulnerable Environment: Alpine

- Docker can be used: ***`docker pull alpine`***
- If a specific release is needed: ***`docker pull alpine:3.9`***
- To run locally*:* ***`docker run -i -t alpine /bin/sh`***

APKBUILD

- Contains the source origin
- Version of the package used within Alpine
- Patches used by Alpine
- Alpine package repositories: https://pkgs.alpinelinux.org/package/edge/community/ppc64le/sox

https://git.alpinelinux.org/aports/tree/community/sox/APKBUILD

path: root/community/sox/APKBUILD

blob: 5689fb7c567c3b680ca3757f39c5992cb6612972 (plain) (blame)

```
# Contributor: Leo <thinkabit.ukim@gmail.com>
# Contributor: Łukasz Jendrysik <scadu@yandex.com>
# Maintainer: Natanael Copa <ncopa@alpinelinux.org>
pkgname=sox
pkgver=14.4.2
pkgrel=5
pkgdesc="The Swiss Army knife of sound processing tools"
url="http://sox.sourceforge.net/"
arch="all"
license="GPL-2.0-or-later LGPL-2.1-or-later"
makedepends="ffmpeg-dev libao-dev libvorbis-dev libogg-dev lame-dev
        libmad-dev bash alsa-lib-dev libsndfile-dev libsamplerate-dev
        file-dev libid3tag-dev flac-dev gsm-dev opusfile-dev libpng-dev
        autoconf automake libtool
        "
subpackages="$pkgname-static $pkgname-doc $pkgname-dev"
source="https://downloads.sourceforge.net/sourceforge/$pkgname/$pkgname-$pkgver.tar.gz
        CVE-2017-15371.patch::https://github.com/mansr/sox/commit/818bdd0.patch
        CVE-2017-11358.patch::https://github.com/mansr/sox/commit/6cb44a4.patch
        CVE-2017-15370.patch::https://github.com/mansr/sox/commit/ef3d8be.patch
        CVE-2017-11332.patch::https://github.com/mansr/sox/commit/7405bca.patch
        CVE-2017-11359.patch::https://github.com/mansr/sox/commit/8b590b3.patch
        CVE-2017-15372.patch::https://github.com/mansr/sox/commit/001c337.patch
        CVE-2017-15642.patch::https://github.com/mansr/sox/commit/0be259e.patch
        CVE-2017-18189.patch
        update-exported-symbol-list.patch
        add-destdir.patch
        cleanup-lsx_malloc.patch
        CVE-2019-8354.patch
        CVE-2019-8355.patch
        CVE-2019-8356.patch
        CVE-2019-8357.patch
        sox-14.4.2-hcom_stopwrite_big_endian_bug_fix.patch
        fix-musl.patch
        "

# secfixes
#   14.4.2-r5:
#     - CVE-2017-13571
#     - CVE-2017-11358
#     - CVE-2017-15370
#     - CVE-2017-11332
#     - CVE-2017-11359
#     - CVE-2017-15372
#     - CVE-2017-13642
```

Installing the Correct Package

- Alpine make minor modifications to an upstream package in order to make it work correctly within Alpine.
- We need to rebuild this package with Debug Symbols enabled
- Alpine uses musl libc instead of glibc and most popular C packages can only be compiled using glibc
- The sox package version available within alpine is 14.4.2-r5 and the vulnerable version of the upstream package is 14.4.2
- The following commit tree can be used:
- https://git.alpinelinux.org/aports/commit/community/sox/APKBUILD?h=3.10-stable&id=304c15f39c09cabeb81bbf573e0faa17c299236f

Modifying APKBUILD to allow Debug Symbols

- Debug symbols provides additional information to a binary such as function names and shared libraries
- helpful while trying to investigate a crash
- Alpine and apktools by default will remove debug information when building packages
- We need to modify our APKBUILD to allow this explicitly

--enable-debug

```
build() {
        cd "$srcdir"/$pkgname-$pkgver
        ./configure \
                --build=$CBUILD \
                --host=$CHOST \
                --enable-debug \
                --prefix=/usr \
                --sysconfdir=/etc \
                --with-dyn-default \
                --with-distro="Alpine Linux" \
                || return 1
        make || return 1
}

package() {
```

!strip

```
# Contributor: Łukasz Jendrysik <scadu@yanc
# Maintainer: Natanael Copa <ncopa@alpinel:
pkgname=sox
pkgver=14.4.2
pkgrel=0
pkgdesc="The Swiss Army knife of sound proc
url="http://sox.sourceforge.net/"
arch="all"
options="!strip"
license="GPL LGPL"
makedepends="ffmpeg-dev libao-dev libvorbis
```

Building an Alpine Package Part 1

- Install Alpine SDK Tools: ***apk -U add alpine-sdk***

- Non root user is needed to build a package within Alpine

```
adduser -D packager && addgroup packager abuild

echo -e "packager\npackager" | passwd packager

echo "packager ALL=(ALL) ALL" >> /etc/sudoers
```

Building an Alpine Package Part 2

- Run the following commands to build an Alpine Package

```
su - packager

abuild-keygen -a -i

abuild -r
```

- Will install all build tools, package dependencies and will fetch the source code of the package

Building an Alpine Package Part 3

This apk can now be installed within alpine by using apk add

apk add /home/packager/packages/home/x86_64/sox-14.4.2-r0.apk --allow-untrusted

```
dde9e804f428:~$ ls
APKBUILD            packages            sox-dynamic.patch  sox-uclibc.patch
dde9e804f428:~$ cd packages/home/x86_64/
dde9e804f428:~/packages/home/x86_64$ ls
APKINDEX.tar.gz         sox-14.4.2-r0.apk       sox-dev-14.4.2-r0.apk  sox-doc-14.4.2-r0.apk
dde9e804f428:~/packages/home/x86_64$
```

Fetching Vulnerable Source Code

- Needed during analysis and debugging process
- Always check that the upstream origin and APKBUILD origin is the same

```
git clone https://git.code.sf.net/p/sox/code sox-code

cd sox-code/

git checkout sox-14.4.2
```

Debugging

- Debuggers are used for dynamic analysis
- A good debugger will have several useful features such as breakpoints, step into, memory searching etc
- We will use GeF which is built on top of vanialla GDB (GNU Debugger)
- Link: https://github.com/hugsy/gef

```
apk add gdb

wget -O ~/.gdbinit-gef.py -q
https://github.com/hugsy/gef/raw/master/gef.py

echo source ~/.gdbinit-gef.py >> ~/.gdbinit
```

Reproducing the Crash

- Start GDB with Sox: ***gdb /usr/bin/sox***
- Link the debug session to the source code: ***set directory /home/packager/packages/sox-code***

```
/home/packager/sox-code # gdb /usr/bin/sox
GNU gdb (GDB) 8.3.1
Copyright (C) 2019 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-alpine-linux-musl".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
GEF for linux ready, type `gef' to start, `gef config' to configure
75 commands loaded for GDB 8.3.1 using Python engine 3.8
[*] 5 commands could not be loaded, run `gef missing` to know why.
Reading symbols from /usr/bin/sox...
gef➢  set directory /home/packager/packages/sox-code
```

Helpful GEF/GDB Commands

break <where>	Set a new breakpoint.
Clear	Delete all breakpoints.
where full	Show call stack, also print the local variables in each frame
Run	Run the program within GDB
Step	Go to next instruction
info files	Show all files part of the program
info sources	Show source directory
info functions	Show all functions

Reproducing the Crash

- https://sourceforge.net/p/sox/bugs/320/ tells us where the heap allocation is
- Review the source code to check where the function allocating to heap is
- Set breakpoint: `break /home/packager/src/sox-14.4.2/src/remix.c:226`
- Run the PoC:

  run --single-threaded /home/packager/sox-code/xmalloc_31_integer_overflow.mp3 -t aiff /dev/null channels 1 rate 16k fade 3 norm

```
gef➢  run --single-threaded /home/packager/sox-code/xmalloc_31_integer_overflow.mp3 -t aiff /dev/null channels 1 rate 16k fade 3 norm
Starting program: /usr/bin/sox --single-threaded /home/packager/sox-code/xmalloc_31_integer_overflow.mp3 -t aiff /dev/null channels 1 rate 16k fade 3 norm
warning: Error disabling address space randomization: Operation not permitted

Breakpoint 1, channels_start (effp=effp@entry=0x7f4c9e5994c0) at remix.c:226
226     remix.c: No such file or directory.
[ Legend: Modified register | Code | Heap | Stack | String ]
$rax   : 0x00007f4c9e4c237c  →  <channels_start+0> push r15
$rbx   : 0x1
$rcx   : 0x0
$rdx   : 0x00007f4c9e5995d8  →  0x0000000000000000
$rsp   : 0x00007fffc3dbe4e0  →  0x0000000000000000
$rbp   : 0x00007f4c9e5994c0  →  0x00007f4c9e599eb0  →  0x0000000000000000
$rsi   : 0x00007f4c9e5995d8  →  0x0000000000000000
$rdi   : 0x00007f4c9e5994c0  →  0x00007f4c9e599eb0  →  0x0000000000000000
$rip   : 0x00007f4c9e4c23a0  →  <channels_start+36> mov r13d, ebx
$r8    : 0x41
$r9    : 0x00007f4c9e5994c0  →  0x00007f4c9e599eb0  →  0x0000000000000000
$r10   : 0x00007fffc3dbe510  →  0x00007f4c9e5994c0  →  0x00007f4c9e599eb0  →  0x0000000000000000
$r11   : 0x0
$r12   : 0x00007f4c9e5995c0  →  0x0000000000000000
$r13   : 0x18
$r14   : 0x00007f4c9e599b28  →  0x40cf400000000000
$r15   : 0x00007fffc3dbe720  →  0x38200d0180000000
$eflags: [zero carry parity adjust sign trap INTERRUPT direction overflow resume virtualx86 identification]
$cs: 0x0033 $ss: 0x002b $ds: 0x0000 $es: 0x0000 $fs: 0x0000 $gs: 0x0000
```

https://gef.readthedocs.io/en/master/commands/heap/

Reproducing the Crash

- We can reproduce the crash
- The vulnerable code is correct/accurate
- Cannot analyse crash properly and cannot examine heap due to Musl and no tools supporting it :(
- However xmalloc being used and program crashing due to out of memory is good indication

```
gef➤  continue
Continuing.
/usr/bin/sox FAIL xmalloc: out of memory
[Inferior 1 (process 2576) exited with code 02]
gef➤
```

Conclusion: Things we are trying to Answer

- Exact input that triggered the crash: https://sourceforge.net/p/sox/bugs/320 ✔️
-
- What vulnerability was triggered: **Integer Overflow** ✔️ (err Maybe)
- Type/Category of the crash: **Heap Overflow** ✔️

Where is the vulnerable code inside the package: *xmalloc.h*, ***lsx_valloc*** - Integer Overflow, heap-buffer-overflow is triggered in ***remix.c line 237*** ✔️

- Exploitable: Not possible to answer in many cases

Further Reading

- https://legacy.gitbook.com/book/dhavalkapil/heap-exploitation
- https://vulncat.fortify.com/en/detail?id=desc.controlflow.cpp.integer_overflow
- https://gef.readthedocs.io/en/master/
- https://github.com/jfoote/exploitable/blob/master/exploitable/lib/rules.py
-